Robótica Inteligente: Práctica I

Memoria técnica de ROS 2, Nav2, RKO-LIO y MoveIt 2

Equipo: _____

Integrantes: _____

Curso 2026–2027

Índice

1. Alcance y método **2**

2. Bloque 2: SLAM y navegación autónoma **2**

2.1. Objetivo y flujo de datos 2

2.2. Interfaces que se deben verificar 2

2.3. Interpretación TF y resultado 3

3. Bloque 3: odometría LiDAR–inercial **3**

3.1. Objetivo y flujo de datos 3

3.2. Interpretación TF y evaluación 4

4. Bloque 4: manipulación con MoveIt 2 **4**

4.1. Objetivo y flujo de datos 4

4.2. Interpretación TF y resultado 5

5. Scripts opcionales y conclusiones **5**

1. Alcance y método

La práctica utiliza ROS 2 Jazzy y Gazebo Harmonic. Los bloques 2, 3 y 4 requieren una inspección del grafo de nodos, interfaces y árbol TF durante la ejecución. Esta memoria recoge la arquitectura que se espera de cada sistema y reserva espacios explícitos para los resultados observados. Las rutas, nombres de nodos y frecuencias se contrastarán con las salidas guardadas en `/ros2_ws/evidencias`; no se presentan como mediciones realizadas.

Para cada fase se ejecuta `bash capturar_interfaces.sh <etapa>`, se abre `ros2 run rqt_graph rqt_graph`, se guarda una captura `grafo.png` y se consulta `ros2 run tf2_tools view_frames`. Los archivos `nodos.txt`, `topicos.txt`, `servicios.txt`, `acciones.txt` y `nodos/*.txt` permiten completar las tablas sin depender únicamente de la imagen del grafo.

2. Bloque 2: SLAM y navegación autónoma

2.1. Objetivo y flujo de datos

Gazebo simula el TurtleBot 4 y sus sensores; el puente `ros_gz_bridge` publica lecturas y reloj en ROS. En la fase SLAM, las mediciones láser y la odometría alimentan `slam_toolbox`. Este nodo estima la transformación entre `map` y `odom` y publica un mapa de ocupación. Al recorrer el almacén con teleoperación, la geometría de pasillos y paredes se incorpora progresivamente. `map_saver_cli` serializa la cuadrícula y sus metadatos en `mapa_laboratorio.pgm` y `mapa_laboratorio.yaml`.

En la fase de navegación se reinicia la simulación sin SLAM y se carga el mapa estático. `map_server` publica `/map`. AMCL compara el láser con el mapa y estima `map` → `odom`; la pose inicial debe suministrarse con *2D Pose Estimate*. Nav2 recibe una meta `NavigateToPose`, calcula una ruta global sobre el mapa de costes, genera órdenes de velocidad con su controlador y las envía al robot. Gazebo integra el movimiento; la nueva odometría y el láser cierran el lazo de realimentación.

Gazebo/sensores → puente ROS → SLAM o AMCL → mapa y TF
meta → BT Navigator → planificador → controlador → velocidad → Gazebo

2.2. Interfaces que se deben verificar

Nodo o componente	Publica	Suscribe	Servicios / acciones
Puente Gazebo–ROS	`/clock`, láser, odometría y estados según configuración	órdenes de velocidad y otros comandos	Comprobar en `node info`
`slam_toolbox` (fase A)	`/map`, `/map_metadata`, TF de mapa	láser, odometría mediante TF	servicios de mapa/serialización; anotar nombres reales
`map_server` (fase B)	`/map`	mapa YAML	`load_map` y `get_map`, si aparecen
`amcl` (fase B)	pose estimada, TF `map`→`odom`	láser, mapa y pose inicial	`set_initial_pose`, según versión
`planner_server`	ruta planificada	mapa de costes y meta	acción `ComputePathToPose`
`controller_server`	velocidad del robot	ruta, odometría y obstáculos	acción `FollowPath`
`bt_navigator`	feedback de navegación	objetivo y estado del sistema	acción `NavigateToPose`

Los nombres de tópicos pueden estar remapeados o bajo un espacio de nombres. La tabla describe funciones esperadas; los nombres exactos se completan con los ficheros de introspección de las dos fases.

Pendiente de evidencia experimental: Grafo de nodos durante SLAM, con explicación de enlaces principales. Archivo esperado: `../ros2_ws/evidencias/bloque2_slam/grafo.png`.

Pendiente de evidencia experimental: Árbol TF de la fase SLAM. Archivo esperado: `../ros2_ws/evidencias/bloque2_slam/frames.pdf`.

Pendiente de evidencia experimental: Grafo de nodos durante navegación autónoma. Archivo esperado: `../ros2_ws/evidencias/bloque2_nav/grafo.png`.

Pendiente de evidencia experimental: Árbol TF de la fase de navegación. Archivo esperado: `../ros2_ws/evidencias/bloque2_nav/frames.pdf`.

Pendiente de evidencia experimental: Añadir una captura del mapa de ocupación construido; el mapa original se guarda como `mapa_laboratorio.pgm` y `mapa_laboratorio.yaml`.

2.3. Interpretación TF y resultado

La cadena esperada es `map` → `odom` → `base_link` → marcos de sensores. La primera transformación corrige la deriva global; la segunda procede de la odometría local; los marcos del láser y del cuerpo describen la geometría del robot. Hay que comprobar en `frames.pdf` autoridad, frecuencia, edad y ausencia de saltos o árboles desconectados. En SLAM, el publicador de `map`→`odom` es el estimador; durante navegación con mapa es AMCL.

Pendiente de evidencia experimental: Añadir la cobertura del mapa, resolución y origen del YAML, una meta alcanzada y una incidencia observada. Comparar la ruta calculada con el movimiento real de Gazebo.

3. Bloque 3: odometría LiDAR–inercial

3.1. Objetivo y flujo de datos

El *rosbag* Oxford Spires reproduce marcas de tiempo, nubes LiDAR e IMU. `rko_lio` combina la predicción inercial entre escaneos con el ajuste geométrico de puntos frente al mapa local. La salida es una pose/odometría y una nube acumulada. `/clock` del reproductor es imprescindible para que los nodos con `use_sim_time:=true` usen el mismo tiempo que los mensajes registrados. RViz muestra la trayectoria y la reconstrucción; no interviene en la estimación.

rosbag (LiDAR + IMU + reloj) → RKO-LIO → odometría + mapa de puntos → RViz

Nodo o componente	Publica	Suscribe	Servicios / acciones
`rosbag2_player`	`/clock`, nube LiDAR, IMU e imágenes	—	controles de reproducción; verificar
`rko_lio`	pose/odometría, TF y nube reconstruida	LiDAR e IMU de la configuración	parámetros de calibración y filtros; verificar
RViz2	visualización y posibles marcadores	odometría, TF y nubes	servicios internos; verificar

Los tópicos del bag dependen de `oxford_spires.yaml`. Se registrarán los nombres y tipos reales con `ros2 bag info`, `ros2 topic list -t` y `ros2 node info`.

Pendiente de evidencia experimental: Grafo del reproductor, RKO-LIO y RViz2. Archivo esperado: `../ros2_ws/evidencias/bloque3/grafo.png`.

Pendiente de evidencia experimental: Árbol TF al procesar Oxford Spires. Archivo esperado: `../ros2_ws/evidencias/bloque3/frames.pdf`.

Pendiente de evidencia experimental: Reconstrucción y trayectoria observadas en RViz2. Archivo esperado: `../ros2_ws/evidencias/bloque3/resultado.png`.

3.2. Interpretación TF y evaluación

Se distinguirá el marco fijo de odometría del marco del LiDAR y de la IMU. Las transformaciones extrínsecas sensor–cuerpo deben ser coherentes con el archivo YAML; las poses cambiantes representan el movimiento estimado. El PDF TF permite comprobar conectividad y qué nodo publica cada relación. Un desajuste temporal entre IMU y LiDAR puede producir saltos, pérdida de escaneos o un mapa borroso; conviene registrar los avisos y la tasa de mensajes, además de una captura del resultado.

Pendiente de evidencia experimental: Indicar duración y tamaño del bag, tópicos exactos, fragmento procesado, avisos de consola y valoración visual de la deriva o nitidez. No atribuir valores numéricos sin medición.

4. Bloque 4: manipulación con MoveIt 2

4.1. Objetivo y flujo de datos

El UR3e simulado tiene seis articulaciones. RViz o el cliente Python envían una pose objetivo del eslabón final a la acción de `move_group`. MoveIt resuelve la cinemática inversa, comprueba límites y colisiones, y OMPL calcula una trayectoria. `ros2_control` recibe puntos articulares mediante un controlador de trayectoria; Gazebo actualiza las articulaciones y publica `/joint_states`. `robot_state_publisher` transforma dichos estados en TF para que el robot visible coincida con el estado simulado.

pose objetivo → MoveIt/planificación → trayectoria articular → controlador → Gazebo
Gazebo → estados articulares → TF y escena de planificación

Nodo o componente	Publica	Suscribe	Servicios / acciones
`move_group`	trayectoria prevista y estado de planificación	`/joint_states`, escena y objetivos	`/move_action`, `/execute_trajectory`
Controlador de trayectoria	estado del controlador	trayectoria articular	acción `FollowJointTrajectory`
`robot_state_publisher`	`/tf`, `/tf_static`	`/joint_states`	parámetros del modelo URDF
Gazebo + puente	estados articulares	comandos del controlador	comprobar interfaces reales
RViz2	meta interactiva	estado, TF y escena	cliente de `/move_action`

Pendiente de evidencia experimental: Grafo de MoveIt, controlador, robot y RViz2. Archivo esperado: `../ros2_ws/evidencias/bloque4/grafo.png`.

Pendiente de evidencia experimental: Árbol TF del UR3e. Archivo esperado: `../ros2_ws/evidencias/bloque4/frames.pdf`.

Pendiente de evidencia experimental: Brazo en la pose ejecutada en Gazebo y RViz2. Archivo esperado: `../ros2_ws/evidencias/bloque4/resultado.png`.

4.2. Interpretación TF y resultado

El árbol debe encadenar `base_link`, los eslabones de hombro, codo y muñeca, y `tool0`; los nombres concretos se confirman con `frames.pdf`. La pose de `tool0` se obtiene componiendo las transformaciones de cada articulación. La meta naranja en RViz representa una configuración deseada; solo cuando la trayectoria se ejecuta y los estados articulares cambian debe afirmarse que Gazebo movió el brazo.

Pendiente de evidencia experimental: Registrar una pose alcanzable (marco, posición, cuaternión), estado de la acción MoveGroup, duración y comparación visual entre pose deseada y ejecutada. Anotar una meta imposible o con colisión y el error observado.

5. Scripts opcionales y conclusiones

`enviar_meta_nav.py` crea una meta `NavigateToPose` en el marco `map` a partir de (x, y, θ). `enviar_pose_ur.py` manda a `MoveGroup` una región de posición y orientación para `tool0`; admite `--plan-only`. Ambos clientes comprueban si la acción está disponible, la aceptación de la meta y el resultado. Se ejecutan después de arrancar sus respectivos sistemas.

Pendiente de evidencia experimental: Incluir el comando exacto y la salida de cada script, y concluir qué se aprendió sobre nodos, acciones y TF a partir de las observaciones reales.

Fuentes

- Guion de la práctica, *Robótica Inteligente: Práctica I*, Universidad de Alicante, 2026–2027.
- https://docs.ros.org/en/jazzy/index.html
- https://turtlebot.github.io/turtlebot4-user-manual/software/turtlebot4_simulator.html
- https://github.com/UniversalRobots/Universal_Robots_ROS2_GZ_Simulation
- https://docs.ros.org/en/ros2_packages/jazzy/api/rko_lio/index.html